

KINGA Engineering Manual — v2.0

Classification: Internal Engineering Reference

Author: Manus AI (commissioned by KINGA Engineering)

Codebase checkpoint: a2e692b0 / August 2026

Status: Authoritative — every behavioural claim is traceable to a specific file or table.

Restricted content notice. This manual deliberately omits fraud-scoring weights, physics calibration constants, financial pricing tiers, and any real claimant personal data. Those values must never appear in contractor-facing documentation, logs shipped to non-privileged roles, or client-visible error messages.

Table of Contents

- [1. System Overview](#)
- [2. Architecture](#)
- [3. Portal Structure and RBAC](#)
- [4. Data Model](#)
- [5. Physics Pipeline](#)
- [6. Fraud Scoring Engine](#)
- [7. Cost Optimisation Engine](#)
- [8. Report Generation Stack](#)
- [9. Benchmark Learning Loop](#)
- [10. Non-Negotiable Invariants — “Never Violate” List](#)
- [11. Development Standards and Conventions](#)
- [12. Testing and QA Requirements](#)
- [13. Deployment and Release Process](#)
- [14. Onboarding Path for a New Engineer](#)
- [15. Glossary](#)
- [16. Gaps and Open Questions](#)

1. System Overview

What KINGA Does

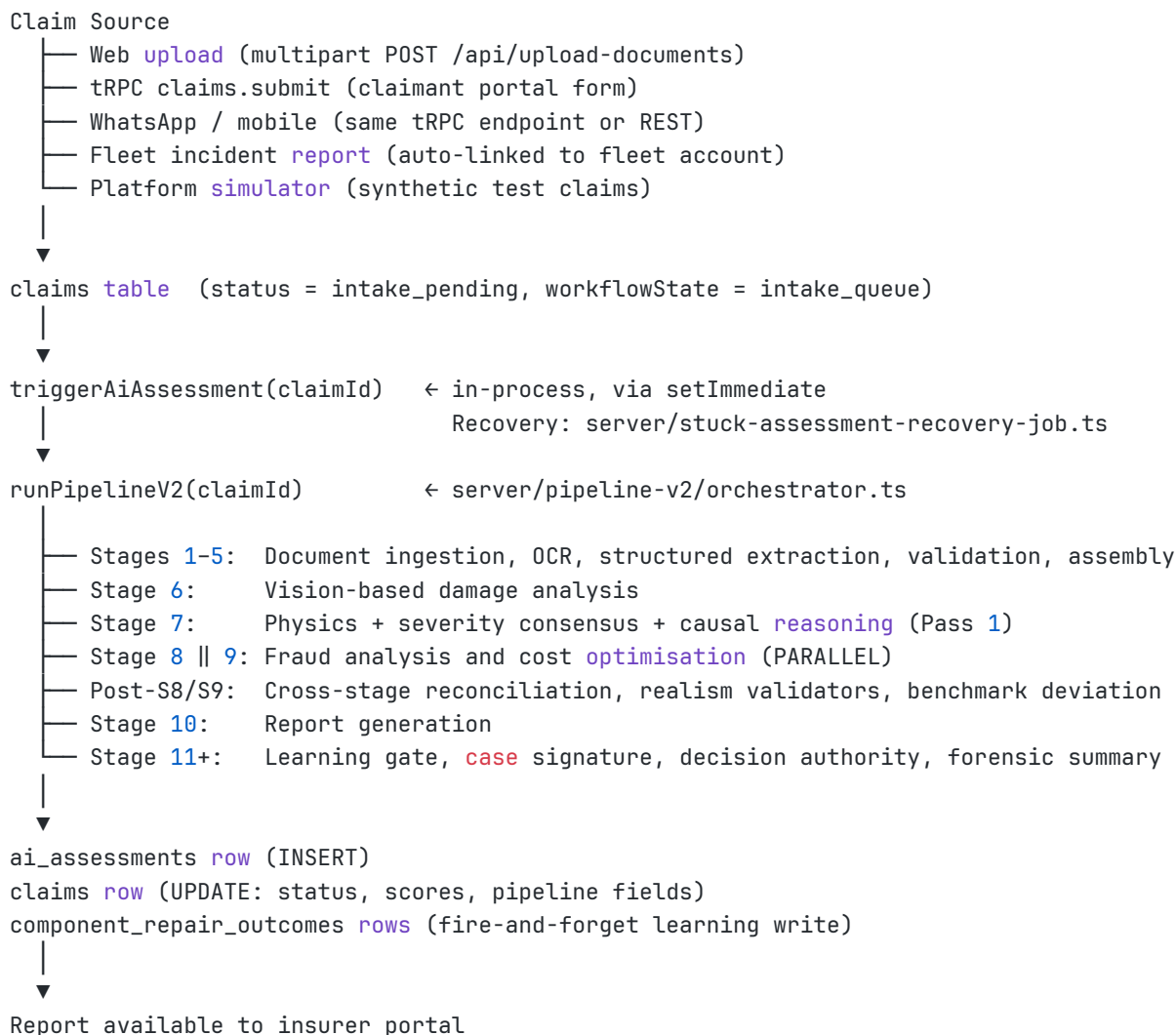
KINGA is a forensic motor insurance claims intelligence platform. Its primary function is to receive a motor insurance claim package — typically a PDF containing a claim form, one or more repair quotations, and damage photographs — run it through a multi-stage AI pipeline, and produce a structured forensic assessment that tells an insurer:

- Whether the physical damage is consistent with the reported incident (physics coherence)
- Whether the claim exhibits fraud indicators and at what risk level (0–100 score, five-level classification)
- What the repair should realistically cost, with parts reconciliation against market benchmarks

- What the recommended decision is: APPROVE, REVIEW, REJECT, ESCALATE, NEGOTIATE, or PROCEED_TO_ASSESSMENT

KINGA is **not** a decision-making system in the legal sense. It produces recommendations and evidence bundles that human adjusters review and act on. Every AI decision is accompanied by a full audit trail including the prompt version, model identifier, input hash, and output snapshot.

High-Level Data Flow



What KINGA Is Not

KINGA does not make legally binding decisions. It does not store claimant biometric data. It does not directly communicate with claimants. It does not handle payment processing. The **claims** table is the single source of truth for claim state; the **ai_assessments** table is the single source of truth for AI pipeline output.

Operating Regions

KINGA is designed to handle claims from multiple countries. Currency and cost normalisation are handled through the **currency_exchange_rates** table and the National Cost Index (NCI) model embedded in the Economic Context Engine (server/db.ts , Phase 2B block). The pipeline currently supports claims denominated in USD, ZAR, ZIG, and other currencies, with explicit currency codes required at every layer.

2. Architecture

Technology Stack

Layer	Technology
Frontend	React 19, Tailwind CSS 4, shadcn/ui
API layer	tRPC 11 (type-safe RPC, no REST routes for feature logic)
Backend runtime	Node.js 22, Express 4
ORM	Drizzle ORM (schema-first, <code>drizzle/schema.ts</code>)
Database	MySQL / TiDB (multi-tenant, single schema)
AI inference	<code>invokeLLM()</code> helper (<code>server/_core/llm.ts</code>) — server-side only
File storage	S3 via <code>storagePut</code> / <code>storageGet</code> (<code>server/storage.ts</code>)
PDF export	Puppeteer-core + system Chromium (<code>/usr/bin/chromium</code>)
Auth	Manus OAuth — session cookie, <code>protectedProcedure</code> injects <code>ctx.user</code>
CI/CD	GitHub Actions (<code>.github/workflows/cicd-pipeline.yml</code>)

Module Map

The codebase is a monorepo. The key directories and their responsibilities are as follows.

`server/pipeline-v2/` contains the entire AI assessment pipeline. The entry point is `orchestrator.ts`, which wires all stages together. Stage files are named `stage-N-*.ts`. Supporting engines (physics, speed inference, fraud, cost, report) live in this directory alongside the orchestrator.

`server/reporting/` contains the three report tiers and their HTML/CSS rendering logic. The design system lives in `server/reporting/templates/kingaDesignSystem.ts`. The PDF renderer is `server/reporting/pdfRenderer.ts`.

`server/rbac.ts` defines all roles (`InsurerRole`), workflow states (`WorkflowState`), and the legal state transition graph (`WORKFLOW_TRANSITIONS`). This is the single source of truth for access control logic.

`server/workflow-engine.ts` is the single gateway for all claim state transitions. No code outside this file should write `workflowState` or `status` directly to the `claims` table without going through `transitionClaim()`.

`server/_core/trpc.ts` defines the tRPC middleware stack including `requireInsurerDomain` (tenant isolation enforcement) and `logTenantIsolationViolation` (fire-and-forget violation audit).

`drizzle/schema.ts` is the canonical database schema. It is the authoritative source for all table definitions, column types, and index declarations.

`client/src/components/` contains all portal layout shells. The shared shell is `KingaPortalShell.tsx`, which implements the Phase 11 visual standard (dark-green hero band, gold separator, white tab bar).

Pipeline Concurrency Model

The pipeline runs in-process (Node.js). A semaphore in `server/db.ts` limits concurrent pipeline executions to one at a time per process instance. This is a deliberate constraint: the pipeline is memory-intensive (Stage 6 vision analysis, Stage

7 physics), and concurrent runs on a single-vCPU container would cause OOM failures. The recovery job (`server/stuck-assessment-recovery-job.ts`) runs every 10 minutes and restarts claims whose `pipeline_heartbeat_at` has not been updated for more than 2 minutes while in a running state.

Orchestrator Contract

The orchestrator (`server/pipeline-v2/orchestrator.ts`) has one inviolable design contract: **it never aborts**. Every stage is wrapped in `runWithTimeout()`. A stage failure degrades gracefully — the failure is logged, a degraded result is synthesised, and execution continues. The final report reflects any degraded stages explicitly. This contract is documented at line 292 of the orchestrator:

“Every stage is wrapped in `runWithTimeout()`; failures degrade gracefully rather than aborting the pipeline.”

The orchestrator exposes two parallel execution points. The first (`Promise.all` at lines ~1338–1456) runs Stage 8 (fraud) and Stage 9 (cost) concurrently. The second (`Promise.all` at lines ~1477–1705) runs the Stage 7b causal reasoning re-run, all cross-stage validators (Stages 35–45), and the turnaround time analysis (Stage 9b) concurrently.

3. Portal Structure and RBAC

Portal Inventory

KINGA exposes fourteen distinct portals, each scoped to a specific role or function. All portals share the `KingaPortalShell` layout component (`client/src/components/KingaPortalShell.tsx`), which enforces the Phase 11 visual standard: a dark-green gradient hero band (`#103A23` → `#1C5C39`), a 2 px gold separator (`#D4A800`), and a white tab bar with Inter Medium typography.

Portal	Layout Component	Primary Role
Insurer Portal	InsurerPortalLayout.tsx	claims_processor , claims_manager
Assessor Portal	AssessorPortalLayout.tsx	assessor_internal , assessor_external
Claimant Portal	ClaimantPortalLayout.tsx	Authenticated claimant
Panel Beater Portal	PanelBeaterPortalLayout.tsx	panel_beater
Engineer Workspace	EngineerWorkspaceLayout.tsx	assessor_internal (specialist)
KINGA Dashboard	KingaDashboardLayout.tsx	admin , platform_super_admin
Platform Admin	PlatformLayout.tsx	admin
Recovery Portal	(KingaPortalShell)	recovery_officer
Risk Manager Portal	(KingaPortalShell)	risk_manager
Executive Portal	(KingaPortalShell)	executive
Fleet Portal	(KingaPortalShell)	Fleet account holder
Broker Portal	(KingaPortalShell)	Broker
Marketplace	(KingaPortalShell)	panel_beater , assessor_external
Claimant Mobile	(KingaPortalShell)	Authenticated claimant (mobile)

Role Definitions

Roles are defined in `server/rbac.ts` as the `InsurerRole` type. The eight insurer-scoped roles are:

Role	Key Permissions
claims_processor	Create claims, view all claims, triage and assign
assessor_internal	View AI assessment, view cost optimisation, conduct internal assessment, view fraud analytics
assessor_external	View AI assessment, view cost optimisation (no fraud analytics)
risk_manager	View all claims, view fraud analytics, approve technical basis
claims_manager	Full visibility, assign assessors, authorise payment, close claims
executive	View-only across all claims and analytics
insurer_admin	Full access including configuration editing
recovery_officer	View all claims, view fraud analytics, add comments

Two platform-level roles exist above the insurer tier: `admin` and `platform_super_admin`. Both are cross-tenant by design and bypass the `requireInsurerDomain` middleware. Cross-tenant admin access is logged to `console.info` but is not written to the `tenant_isolation_violations` table (that table is reserved for genuine violations).

Tenant Isolation Enforcement

All insurer-facing tRPC procedures are wrapped in the `requireInsurerDomain` middleware (`server/_core/trpc.ts`). This middleware:

1. Rejects unauthenticated requests with `UNAUTHORIZED` .
2. Allows `admin` and `platform_super_admin` through with their own `tenantId` (or null for cross-tenant).
3. For all other roles, requires `ctx.user.tenantId` to be non-null. A null `tenantId` on an authenticated non-admin user is treated as a tenant isolation violation: the request is rejected with `FORBIDDEN` , and the event is asynchronously written to the `tenant_isolation_violations` table (fire-and-forget — the write failure never blocks the exception).

The `requireTenantScope()` helper function (`server/_core/trpc.ts` , line 367) is the canonical call-site for extracting the scoped `tenantId` within a procedure body. Non-admin callers receive their own `tenantId` ; platform admins receive `null` (cross-tenant).

Workflow State Machine

The claim lifecycle is governed by the `WorkflowState` type and the `WORKFLOW_TRANSITIONS` graph, both defined in `server/rbac.ts` . All transitions must go through `transitionClaim()` in `server/workflow-engine.ts` . Direct writes to `workflowState` outside the workflow engine are a governance violation.

```
intake_queue → created → assigned → under_assessment
                                     |
                                     internal_review
                                     |
                                     technical_approval
                                     |
                                     financial_decision
                                     |
                                     payment_authorized → closed
                                     |
                                     disputed → internal_review (restart)
```

The workflow engine enforces segregation of duties: a user who performed the `under_assessment` stage cannot also perform `technical_approval` or `financial_decision` . This is tracked in the `claim_involvement_tracking` table.

SLA Visibility

The `SLADeadlineChip` component (`client/src/components/portal/SLADeadlineChip.tsx`) is the shared SLA deadline indicator. It accepts `createdAt` , `deadline` , or `hoursRemaining` as props and renders a colour-coded chip (green → amber → red) based on time remaining. The default SLA for intake queue claims is 72 hours. The chip is used in `IntakeQueueTab.tsx` and should be used in any new queue view that exposes SLA-sensitive claims.

4. Data Model

Core Tables

The canonical schema is `drizzle/schema.ts` . The following tables are central to understanding the system.

claims is the master claim record. It holds claim identity, vehicle data, incident data, pipeline state, workflow state, fraud scores, and all pipeline backfill fields. Key design decisions:

- **status** (legacy enum, 19 values) and **workflowState** (governance enum, 13 values) coexist for backward compatibility. The workflow engine writes both on every transition.
- **claimantStatedSpeedKmh** is **immutable after first write**. It is populated once by Stage 3 and must never be overwritten by any later stage. **estimatedSpeedKmh** is the mutable consensus output. These two fields must never be conflated. A defensive guard column **claimantSpeedNeedsVerification** is set to 1 when a re-run detects that **claimantStatedSpeedKmh** is NULL but **estimatedSpeedKmh** has already been written by a prior consensus run.
- **pipelineHeartbeatAt** is updated every 30 seconds by the running pipeline stage. The recovery job uses this to detect stuck pipelines.
- **pipelineRunUuid** links the claim to its active **pipeline_runs** row, enabling the resume-from-checkpoint mechanism.
- **fraudRiskLevel** is a legacy enum with five values: **low**, **medium**, **moderate**, **high**. Note that **moderate** and **medium** are both present — this is a known inconsistency (TD-001 in the v1.0 baseline certification report). The canonical fraud level is stored in the **ai_assessments** row; the **claims** row value is a denormalised summary for fast querying.

ai_assessments holds the complete output of a single pipeline run. Key columns include **physicsAnalysisJson**, **fraudAnalysisJson**, **costAnalysisJson**, **reportJson**, **physicsTruthJson**, **claimTruthJson**, **ifeResultJson**, **doeResultJson**, **felVersionSnapshotJson**, and **reportSignalsJson**. These are all JSON blobs. The **ai_assessments** row is the authoritative record of what the pipeline computed; the **claims** row fields are denormalised summaries for querying and display.

component_repair_outcomes is the benchmark learning table. It receives one row per damaged component per completed claim, written at **ANALYSIS_COMPLETE** by the fire-and-forget learning path in **server/db.ts** (lines ~2665–2720). The G-1 fraud guard prevents claims with **fraud_risk_score** ≥ 50 from writing to this table. See Section 9 for the full write path.

currency_exchange_rates stores exchange rates to USD. All cost values in the pipeline are normalised to USD internally. The **currencyCode** field on the **claims** row and the **currency** field on **Stage90Output** carry the original quote currency through to the report.

cost_components stores per-source cost breakdowns (panel beater quote, assessor report, AI estimate, final approved) for historical claims used in benchmark computation.

workflow_audit_trail is an append-only log of every claim state transition, written by the workflow engine.

tenant_isolation_violations is an append-only log of every FORBIDDEN event raised by the **requireInsurerDomain** middleware.

pipeline_runs and **pipeline_stage_results** provide per-run, per-stage observability. The **loadCompletedStages(runUuid)** function in **server/pipeline-v2/stageResultCache.ts** reads these tables to enable pipeline resume after a crash.

Read/Write Ownership

The following table documents which layer owns writes to the most sensitive fields.

Field	Owner	Never Written By
<code>claims.claimantStatedSpeedKmh</code>	Stage 3 (first run only)	Any later stage, any UI action
<code>claims.workflowState</code>	<code>workflow-engine.ts</code> only	Direct Drizzle updates in routers
<code>claims.fraudRiskScore</code>	Pipeline final write	UI (display only)
<code>ai_assessments.*</code>	Pipeline final write	Any UI mutation
<code>component_repair_outcomes.*</code>	<code>repairReplaceEngine.ts</code>	Any direct SQL outside that module
<code>tenant_isolation_violations.*</code>	<code>_core/trpc.ts</code> middleware	Any application code

Immutable and Append-Only Tables

Several tables are designed to be immutable or append-only:

- `workflow_audit_trail` — append-only; rows are never updated or deleted.
- `tenant_isolation_violations` — append-only.
- `pipeline_stage_results` — append-only; each stage result is a new row.
- `component_repair_outcomes` — the finalization path uses `INSERT IGNORE` (first write wins); the adjuster correction path uses `INSERT ... ON DUPLICATE KEY UPDATE` (adjuster wins). Neither path deletes rows.
- `ai_assessments` — each pipeline run produces a new row. Re-runs do not overwrite prior assessments.

The `isImmutable` column (`tinyint` , default 1) appears on certain reference tables (e.g., `vehicle_geometry_profiles`) to signal that rows should not be modified after seeding.

5. Physics Pipeline

Purpose and Scope

The physics pipeline is the most technically distinctive component of KINGA. Its purpose is to independently reconstruct the physical parameters of the accident — primarily impact speed and force — from the physical evidence in the claim documents (damage photographs, crush depth measurements, component damage patterns) and compare those reconstructed parameters against the claimant’s stated account. Inconsistencies between the physics reconstruction and the stated account are surfaced as adjuster flags, not as automated fraud scores.

The physics pipeline spans Stages 6, 6.5A, 6.5B, 6.5C, 7, 7b, 7d, and the integrity engine. It is implemented across the following files:

- `server/pipeline-v2/stage-6-5a-vge.ts` — Vision Geometry Engine (VGE): scale calibration from reference objects in damage photos
- `server/pipeline-v2/stage-6-5b-vgr.ts` — Vision Geometry Reconciler (VGR): cross-image crush depth consensus
- `server/pipeline-v2/stage-6-5c-slpe.ts` — Structural Load Path Estimator (SLPE): component-level deformation energy
- `server/pipeline-v2/stage-7-physics.ts` — Physics Analysis Engine: orchestrates all physics sub-engines
- `server/pipeline-v2/speedInferenceEnsemble.ts` — Speed Inference Ensemble: M1–M7 multi-method speed consensus

- `server/pipeline-v2/damageClassificationEngine.ts` — Damage Classification Engine: crush depth and airbag consistency checks
- `server/pipeline-v2/stage-integrity.ts` — Physics Integrity Engine: 10 cross-measurement contradiction checks
- `server/pipeline-v2/physicsTruth.ts` — PhysicsTruth object builder: canonical physics output record
- `server/accidentPhysics.ts` — Legacy physics engine (still called by Stage 7 via dynamic import)

Speed Inference Ensemble (M1–M7)

The speed inference ensemble (`server/pipeline-v2/speedInferenceEnsemble.ts`) computes a consensus impact speed from up to seven independent methods. The consensus is a weighted mean of all methods that successfully ran.

Method	Basis	Status	Weight
M1 — Campbell's Stiffness Formula	Crush depth × vehicle stiffness coefficient	Active	High (varies by crush depth confidence)
M2 — Energy-Momentum Balance	Repair cost as energy proxy	Disabled	0 — repair cost is market-dependent
M3 — Impulse-Momentum	Damage contact patch area	Disabled	0 — requires primary contact patch area
M4 — Deployment Threshold	FMVSS 208 / Euro NCAP restraint system response	Active	Point estimate contributor, not floor
M5 — Vision Deformation Energy	Component-level energy sum from Stage 6	Active	0.10 (reduced from 0. ⁴⁵ / _{0.65} in KINGA-AUDIT-2026-07)
M6 — Severity-Anchored Inference	Published crash energy signature data (SAE 2002-01-0547)	Active	0.20 (reduced from 0.35 in KINGA-AUDIT-2026-07)
M7 — Claimant-Stated Speed	Driver's stated speed from claim documents	Active	0.30 (when plausible)

Critical design notes on M1/M5 correlation. M1 (Campbell) and the original M5 Path A both used the same `crushDepthM` input. The prior weights (0.45 for M5) overstated independence between the two methods, effectively double-counting the crush depth measurement. KINGA-AUDIT-2026-07 corrected this by reducing M5's weight to 0.10 and removing M5 Path A. M1 is now the single Campbell-based method.

M4 is a weighted contributor, not a floor. An airbag deployment threshold (approximately 28 km/h for frontal airbags, 18 km/h for pretensioners) establishes a lower bound on impact speed. The prior implementation treated M4 as a hard floor that could silently override the consensus. The current implementation treats M4 as a point estimate contributor with explicit contradiction reporting when the consensus falls below the deployment threshold.

M7 reads from `claimantStatedSpeedKmh`, never from `estimatedSpeedKmh`. The ensemble reads the claimant-stated speed from `claims.claimantStatedSpeedKmh` (the immutable field). It never reads from `claims.estimatedSpeedKmh` (the mutable consensus output). This separation is enforced by the schema design and documented in the column comment in `drizzle/schema.ts`.

Vision Geometry Engine (VGE)

The VGE (`server/pipeline-v2/stage-6-5a-vge.ts`) converts pixel measurements in damage photographs to physical dimensions using known reference objects detected in the images. The VVCS coordinate convention is: origin at front axle centreline at ground level, X positive forward, Y positive left, Z positive up, all coordinates in millimetres.

Reference objects are tiered by reliability:

- **Tier 1** (highest): wheel/tyre, licence plate
- **Tier 2** (useful): headlamp spacing, grille width, bonnet width
- **Tier 3** (supporting only): badges, door handles, trim lines

The VGE degrades gracefully: if no vehicle profile exists in the database, it attempts wheel detection using the tyre specification from the claim. If no reference objects are detected, it returns `null` and Stage 7 falls back to the raw LLM crush depth estimate. If the wheel ellipse aspect ratio is below 0.4 (extreme perspective), calibration confidence is set to LOW and no correction is applied.

The `visionSourceReliability` field on `Stage6Output` communicates the reliability of the vision source to Stage 7. When reliability is `LOW` or `NONE`, Stage 7 nulls out `visionCrushDepthM` and does not use it as a point estimate in M1.

Anti-Circularity Design

A critical design constraint governs the crush depth consistency check in `server/pipeline-v2/damageClassificationEngine.ts`. The consensus speed (M1–M6) is derived substantially from crush depth via M1 and M5. Comparing crush depth against a range derived from the consensus speed would therefore be circular — the check would always pass because the crush depth was used to compute the speed that defines the expected range.

The solution is documented at lines 314–338 of `damageClassificationEngine.ts`. The primary anchor for the crush depth consistency check is the **Stage 6 severity/structural rating** — an independent holistic judgment from vision analysis of damage photographs, distinct from the specific crush depth measurement and not derived from any claimant input. The consensus speed is used only as a clearly labelled `CONSENSUS_FALLBACK` anchor when no severity rating is available. The claimant-stated speed is **never** used as an anchor for this check — it is a comparison input only and must not flow into any evidence-grading calculation.

This anti-circularity principle is a standing design rule. Any future validation logic that compares a measurement against a range must verify that the range is not derived from the measurement being validated.

Physics Integrity Engine

The Physics Integrity Engine (`server/pipeline-v2/stage-integrity.ts`) performs ten cross-measurement contradiction checks on the fully assembled `PhysicsTruth` object. Each check produces a flag with a severity (`CRITICAL`, `WARNING`, or `INFO`), a description, affected measurements, a recommendation, and numeric evidence values.

Check	Description
INT-01	Speed vs crush depth consistency (Campbell formula cross-check)
INT-02	Kinetic energy vs deformation energy balance
INT-03	VGR cross-image agreement
INT-04	Speed ensemble method divergence
INT-05	Airbag deployment vs speed threshold
INT-06	Claimed speed vs physics-derived speed
INT-07	SLPE structural integrity risk vs speed
INT-08	Data quality score vs decision confidence
INT-09	Crush depth source quality vs claim value
INT-10	Deformation efficiency factor plausibility

A `CRITICAL` flag means the contradiction is severe enough to invalidate the physics conclusion. The claim must be reviewed before a decision is made. The integrity engine result is stored in `ai_assessments.physicsTruthJson` and surfaced in the Forensic Decision Report.

Stage Input Guards

Before each of Stages 6, 7, 8, and 9, the orchestrator calls the corresponding guard function from `server/pipeline-v2/stageInputGuards.ts`. Guards never throw or block the pipeline. They return a `StageInputReport` listing missing or low-confidence fields, and produce a plain-English `DATA_GAPS_WARNING` preamble that is prepended to the LLM system prompt for that stage. This ensures the model explicitly flags missing data rather than silently inferring values.

6. Fraud Scoring Engine

Purpose

The fraud scoring engine (`server/fraud-scoring.ts`) produces a composite fraud score (0–100) and a five-level risk classification for each claim. The score is a weighted sum of signals across ten indicator categories. The engine is deterministic: identical inputs always produce identical outputs. It does not learn or adapt at runtime; calibration changes require a code change with a governance review.

Fraud Score Bands

The canonical fraud score bands are defined in `shared/fraudScoring.ts` and documented in `docs/KINGA-FRAUD-SCORING-STANDARD.md`. All platform components — pipeline stages, enforcement engines, report generators, API serialisers, and UI helpers — must delegate to `shared/fraudScoring.ts`. No component may define its own score-to-level mapping. This rule was introduced to resolve a historical drift where three independent implementations existed with inconsistent boundary definitions.

Level	Score Range	Routing Consequence
minimal	0–19	Straight-through processing
low	20–39	Soft-review routing
moderate	40–60	Assessor review required; automated approval suspended
high	61–80	Senior assessor; enhanced audit logging
elevated	81–100	Fraud investigation unit; potential regulatory reporting

The score = 20 boundary is the most historically inconsistent point in the codebase. All implementations must return "low" for score = 20.

Note on fraud-scoring.ts vs shared/fraudScoring.ts . The fraud-scoring.ts file contains a legacy rawRiskLevel() function with different thresholds (76/56/36/16) that pre-dates the governance standard. The canonical thresholds are in shared/fraudScoring.ts . Any new code reading a fraud level must use the shared utility.

Input Categories

The FraudScoringInput interface (server/fraud-scoring.ts , line 95) accepts signals across ten categories. The categories and their key signals are:

Category	Key Signals
physics	Damage consistency score, impossible damage patterns, severity mismatch, staged accident indicators
claimant	Policy age, submission delay, previous claims count, driver licence status, driver age
staged	Estimated speed, number of injury claims, witnesses, dashcam footage, police report, incident hour
panelBeater	Quote similarity score, parts inflation %, labour inflation %, replacement-to-repair ratio
assessor	Rubber-stamping score, bias score, collusion score, average turnaround hours
collusion	Triad repeat count, shared contacts with panel beater or assessor
documents	Photo metadata score, reused photo score, document consistency score, OCR confidence
costs	Quoted total vs AI estimate, repair-to-value ratio, overpriced parts count
vehicle	Vehicle age, ownership transfer days before claim, VIN mismatch, previous accident count
timing	Weekend/holiday submission, rapid resubmission, policy lapse notice days before claim

The mlResult field accepts an optional ML model output (fraud probability, ownership risk score, staged accident indicators, driver profile risk score) that is blended into the composite score when present.

Cross-Validation as a QA Layer

The cross-validation engine (server/cross-validation.ts) is invoked from server/assessment-processor.ts (line 2147) and compares the list of quoted parts (from the panel beater quotation) against the damage visible in photographs. It produces a CrossValidationReport with three counts:

- **Confirmed:** Parts quoted and visible in photos.
- **Quoted not visible:** Parts quoted but not visible in photos (suspicious).
- **Visible not quoted:** Damage visible in photos but not quoted (under-quoting).

The `overallRiskScore` (0–100) and `overallRiskLevel` from the cross-validation report are fed into the fraud analysis as additional signals (line 2204 of `assessment-processor.ts`).

To run cross-validation manually against a specific claim, call `crossValidateQuotesVsPhotos(quotedParts, damagePhotoUrls)` directly in a test script. A **false pass** occurs when the photo analysis returns low confidence (e.g., blurry or dark photos) and the engine defaults to “confirmed” for all quoted parts. The `photoMetadataScore` signal in the fraud input captures this: low OCR/photo confidence reduces the weight of the cross-validation result.

7. Cost Optimisation Engine

Purpose

The cost optimisation engine (`server/pipeline-v2/stage-9-cost.ts`) produces a validated cost estimate for the claim, reconciles it against the panel beater quotation and any assessor report, and recommends a cost decision (APPROVE, NEGOTIATE, REJECT, ESCALATE, or PROCEED_TO_ASSESSMENT).

Two Operating Modes

The engine operates in two modes depending on whether a physical assessment has been completed.

PRE_ASSESSMENT mode is used when no assessor report has been submitted. The AI estimate is used as a benchmark only. The `optimised_cost_usd` is set to the lowest submitted quote. The cost decision is PROCEED_TO_ASSESSMENT unless the quote is within the AI benchmark range.

POST_ASSESSMENT mode is used when an assessor report is available. The assessor’s validated cost is used as the primary basis. The AI estimate is used as a cross-check. The `cost_basis` field on the `CostDecisionResult` is set to `"assessor_validated"`.

The mode is selected at line 900 of `stage-9-cost.ts` based on whether `agreedCostUsd` is non-null.

Repair-vs-Replace Engine

The `RepairReplaceEngine` (`server/pipeline-v2/repairReplaceEngine.ts`) computes a `repairProbability` score (0–100) for each detected component using four weighted signals:

Signal	Weight
Damage severity (from Stage 6)	40%
Component category (structural / panel / glass / mechanical)	25%
Vehicle context (age, make tier, market segment)	20%
Learning DB history (from <code>component_repair_outcomes</code>)	15%

Score interpretation: ≥ 66 → Repair recommended; 40–65 → Uncertain (physical inspection required); < 40 → Replace recommended. The engine is deliberately probabilistic and never makes a hard decision.

Benchmark Query

The `getComponentBenchmarks()` function (`server/db.ts` , line ~4480) queries `component_repair_outcomes` for p25/median/p75 cost benchmarks per component. It first attempts a make-specific query; if no rows exist for the specific make, it falls back to all makes. It returns `null` for a component when no historical data exists at all. Stage 9 calls this function to populate the `benchmarkDeviation` field on each component's cost estimate.

Currency Architecture

All cost values in the pipeline are normalised to USD internally. The `currencyCode` field on the `claims` row and the `currency` field on `Stage9Output` carry the original quote currency through to the report. The National Cost Index (NCI) model in the Economic Context Engine (`server/db.ts` , Phase 2B block) adjusts cost benchmarks for regional purchasing power. Every currency-bearing value must carry an explicit, unambiguous currency unit at every layer it passes through. Implicit “assumed currency” is a class-1 invariant violation (see Section 10).

8. Report Generation Stack

Three Report Tiers

KINGA produces three tiers of report, each with a different audience and information depth.

Tier	Report Key	Audience	Contents
Tier 1 — Claim Intelligence Report	<code>claim.intelligence_report</code>	Claims processor, assessor	Full AI assessment, physics analysis, fraud signals, cost breakdown
Tier 2 — Forensic Decision Report	<code>claim.forensic_decision_report</code>	Risk manager, claims manager	Physics integrity flags, fraud evidence, cost decision, recommendation
Tier 3 — Executive Summary	<code>claim.executive_summary</code>	Executive, insurer admin	KPI summary, decision, key flags only

The report dispatcher (`server/reporting/reportDefinitions.ts`) maps each report key to the roles permitted to access it. The `REPORT_ACCESS` map is the authoritative access control list for report generation. Any new report type must be added to this map before it can be served.

HTML/PDF Rendering Pipeline

Reports are rendered as HTML using Handlebars-style templates and then converted to PDF using Puppeteer-core + system Chromium (`/usr/bin/chromium`). The rendering pipeline is:

1. The report data object (assembled from `ai_assessments` and `claims` rows) is passed to the appropriate template function in `server/reporting/`.
2. The template function produces an HTML string using the KINGA design system (`server/reporting/templates/kingaDesignSystem.ts`).
3. The HTML string is passed to `renderToPdf()` in `server/reporting/pdfRenderer.ts`.
4. Puppeteer launches a headless Chromium instance, loads the HTML, and renders to PDF.
5. The PDF buffer is uploaded to S3 via `storagePut()` and the URL is stored in `ai_assessments.reportPdfUrl`.

Why HTML tables, not CSS flexbox/grid, for report layouts. The KINGA design system uses HTML `<table>` elements for multi-column report layouts. This is a deliberate choice, not a legacy oversight. Puppeteer's PDF rendering engine has historically produced inconsistent results with CSS flexbox and grid across page breaks, particularly for tables that span multiple pages. HTML tables with explicit column widths produce deterministic pagination. This rule is documented in `server/reporting/templates/kingaDesignSystem.ts`.

PDF Renderer Design Constraints

The PDF renderer (`server/reporting/pdfRenderer.ts`) enforces three constraints introduced in the M-02/M-06 fix:

1. **Bounded concurrency** — at most 3 concurrent Puppeteer renders via `p-limit`. This prevents OOM failures on the single-vCPU container.
2. **Faster page load** — `waitUntil: "domcontentloaded"` instead of `"networkidle0"`. This eliminates the 30-second timeout that previously caused render failures when external resources were unreachable.
3. **Retry loop** — up to 2 retries with 2-second and 4-second backoff on transient failures (timeout, navigation, protocol errors, target closed).

Report Access Control

The `REPORT_ACCESS` map in `server/reporting/reportDefinitions.ts` defines which roles may access which report types. The key `"risk_manager_portfolio"` is a non-standard key that does not match the dot-notation convention used by all other keys. This is documented as TD-004 in the v1.0 baseline certification report and is scheduled for correction in v1.1. Until corrected, the `REPORT_ACCESS` check for this key will not match any standard report request using the dot-notation dispatcher.

Report Chrome Isolation

Report HTML must never include application chrome — navigation bars, development banners, debug overlays, or any React component that is part of the live application shell. The PDF generation path renders HTML in an isolated Puppeteer context that has no access to the live application. Any component that conditionally renders based on `process.env.NODE_ENV` must be explicitly excluded from the report template. This is a class-1 invariant (see Section 10).

9. Benchmark Learning Loop

What It Is

The benchmark learning loop is the mechanism by which KINGA improves its repair cost estimates over time. When a claim is finalised with a confirmed repair or replacement outcome, the per-component cost data is written to the `component_repair_outcomes` table. Future claims for the same component type and vehicle make query this table to calibrate the AI cost estimate against historical adjuster-confirmed outcomes.

What Table It Writes To

The learning loop writes to `component_repair_outcomes` (`drizzle/schema.ts`). Each row records: `claimId`, `componentName`, `componentCategory`, `vehicleMake`, `vehicleModel`, `vehicleYear`, `outcome` (repair/replace/write_off), `repairCostUsd`, `replaceCostUsd`, `finalCostUsd`, `fraudScore`, `tenantId`, and `createdAt`.

The Full Write Path

The write path has two branches:

Branch 1 — Automatic Finalization Write (fire-and-forget). This is the primary branch. It is triggered automatically when the pipeline reaches `ANALYSIS_COMPLETE` status. The write is performed in `server/db.ts` at line 2664 inside the `recordFinalizedOutcome()` function. The write is fire-and-forget: it runs in a `setImmediate` callback and its promise is not awaited. A write failure is logged to `console.error` but never surfaces to the caller or blocks the pipeline.

Branch 2 — Adjuster Correction Write (UPSERT). This is the secondary branch. It is triggered when an adjuster explicitly confirms or overrides a repair/replace decision via `trpc.repairReplace.recordOutcome`. The write is performed in `server/pipeline-v2/repairReplaceEngine.ts` at line 257 inside `recordAdjusterOutcome()`. This branch uses `INSERT ... ON DUPLICATE KEY UPDATE` (adjuster wins). The router returns `{ recorded: boolean; skippedReason?: string }` so the UI can acknowledge a G-1 skip.

The G-1 Fraud Exclusion Guard

Both write branches are gated by the G-1 fraud exclusion guard. Claims with `fraud_risk_score ≥ 50` (on the 0–100 scale) are excluded from the learning table. The guard is implemented in `repairReplaceEngine.ts` and applied before any INSERT is executed. If the guard query fails, the function skips the write rather than proceeding unguarded.

Why this guard exists. A fraudulent or inflated claim produces cost data that does not reflect genuine repair economics. If allowed into the learning table, it would systematically bias future cost benchmarks upward, causing the AI to approve inflated quotes. The guard threshold of 50 is consistent with the threshold used in `analytics-db.ts` for high-risk claim identification.

Historical Failure: The Unactivated Write Path

Standing rule derived from this incident: Any pipeline side-effect that depends on a UI trigger must have an automated fallback trigger, or a test that fails if the UI path is never exercised.

The investigation report `docs/KINGA_LEARNING_TABLE_INVESTIGATION_REPORT.md` (KINGA-INV-001, 2026-07-31) documents the following confirmed facts:

1. As of 2026-07-31, the `component_repair_outcomes` table contained **0 rows** in production.
2. There was exactly one write call site: `recordAdjusterOutcome()` in `repairReplaceEngine.ts`. An earlier diagnosis document had incorrectly named `costLearningRecorder.ts` as the writer; that module writes to a different table (`cost_learning_records`).
3. The write path was structurally functional — a live end-to-end test confirmed the INSERT executes successfully.
4. The table was empty because the write was **never triggered automatically**. It required an explicit adjuster annotation via `trpc.repairReplace.recordOutcome`, and no UI component called this procedure during the normal claim lifecycle.

The resolution was to implement Branch 1 (the automatic finalization write in `server/db.ts`) so that the learning loop activates without requiring a UI action. The G-1 guard was implemented at the same time, before any data began accumulating.

A new engineer should be able to explain this incident before touching the pipeline unsupervised. The lesson is not that the code was broken — the write path was functional. The lesson is that a side-effect that depends exclusively on a UI trigger is invisible to the pipeline test suite and will silently fail to accumulate data in production.

10. Non-Negotiable Invariants — “Never Violate” List

The following invariants are derived from confirmed past incidents and first-principles system requirements. Each has a

one-line rationale. Violating any of these invariants constitutes a P0 defect regardless of the feature context.

#	Invariant	Rationale
INV-01	Claimant-stated values (e.g., <code>claimantStatedSpeedKmh</code>) are never overwritten by pipeline-derived consensus values. They are separate fields, always.	The claimant's stated account is evidence. Overwriting it with a derived value destroys the audit trail and makes it impossible to detect inconsistency between the account and the physics.
INV-02	No pipeline stage may silently drop or swallow an extraction or processing failure. Every failure must be logged, surfaced, and block finalization where correctness depends on it.	Silent defaults substituted for missing data produce plausible-looking but incorrect assessments. The <code>DATA_GAPS_WARNING</code> preamble mechanism exists precisely to prevent this.
INV-03	No claim data may cross tenant boundaries under any code path, including shared or reusable components.	Multi-tenancy is a contractual and regulatory obligation. Cross-tenant data leakage is a reportable breach. The <code>requireInsurerDomain</code> middleware and <code>tenant_isolation_violations</code> table enforce this at the API layer.
INV-04	No cost or scoring composite may be computed by aggregating across claims without explicit claim-scoped filtering.	Aggregating across claims without a <code>WHERE claimId = ?</code> or <code>WHERE tenantId = ?</code> filter produces cross-claim contamination — a score or cost that is influenced by data from unrelated claims. This is both a correctness defect and a tenant isolation violation.
INV-05	Every currency-bearing value must carry an explicit, unambiguous currency unit at every layer it passes through. No implicit “assumed currency.”	Implicit currency assumptions cause systematic mispricing when claims are denominated in non-USD currencies. The NCI model and currency normalisation path exist to handle this correctly; bypassing them with an implicit assumption defeats the entire mechanism.
INV-06	The benchmark learning loop must never write outcomes for a claim with a disqualifying fraud score (≥ 50).	Fraudulent or inflated claims produce cost data that does not reflect genuine repair economics. Allowing them into the learning table biases future benchmarks upward. The G-1 guard implements this.
INV-07	Any validation logic must not depend on a value derived from the thing it is validating (the circular crush-depth bug class).	Circular validation always passes because the expected range is derived from the measurement being validated. The anti-circularity design in <code>damageClassificationEngine.ts</code> (lines 308–338) documents the correct pattern.
INV-08	Report output must never include application chrome (navigation bars, development banners, debug overlays). PDF generation paths must be tested in isolation from the live application shell.	Application chrome in a PDF report is a presentation defect that undermines the forensic credibility of the document. It also indicates that the report template is not isolated from the live application, which creates a dependency on the application's runtime state.
INV-09	Fraud-scoring weights, physics calibration constants, and financial configuration must never appear in contractor-facing documentation, logs shipped to non-	These values are proprietary and competitively sensitive. Exposing them enables gaming of the scoring system. This manual deliberately omits them.

#	Invariant	Rationale
	privileged roles, or client-visible error messages.	
INV-10	All claim state transitions must go through <code>transitionClaim()</code> in <code>server/workflow-engine.ts</code>. Direct writes to <code>workflowState</code> outside the workflow engine are a governance violation.	The workflow engine enforces segregation of duties, role permissions, and audit trail logging. Bypassing it produces an inconsistent audit trail and may allow a single user to complete the full claim lifecycle without oversight.

11. Development Standards and Conventions

Type Contract Enforcement

KINGA uses TypeScript throughout the server layer. The intended standard is static typing enforced at compile time, with runtime validation at API boundaries (tRPC input schemas via Zod). In practice, the codebase has a significant number of files annotated with `// @ts-nocheck` at the top — 90 server files as of the v1.0 baseline audit. This annotation suppresses all TypeScript errors in the file.

The `@ts-nocheck` annotation is a technical debt marker, not an accepted practice. It exists because the codebase grew rapidly and some modules were written before strict typing conventions were established. The correct approach is to remove `@ts-nocheck` from a file only after resolving all type errors in that file, not to leave it in place indefinitely.

What “enforced” means concretely in this codebase:

- In files without `@ts-nocheck`: TypeScript errors are compile-time failures. The CI pipeline runs `pnpm tsc --noEmit` as a hard gate.
- In files with `@ts-nocheck`: TypeScript errors are suppressed. The file is effectively untyped at compile time. Runtime type errors in these files will not be caught until the code executes.
- At API boundaries: tRPC input schemas use Zod for runtime validation. Invalid inputs are rejected before reaching procedure logic.
- For pipeline stage outputs: the `StageNOutput` types in `server/pipeline-v2/types.ts` define the contracts between stages. These types are enforced in files without `@ts-nocheck`; in files with `@ts-nocheck`, they are documentation only.

Any new code added to the codebase must be written without `@ts-nocheck`. Existing `@ts-nocheck` files should be cleaned up incrementally as part of regular maintenance.

Error Handling Standard

The KINGA error handling standard has one rule: **no silent catches**. A catch block that swallows an error without logging it or propagating it is a defect.

A compliant error-handling block in the pipeline context looks like this (from `server/pipeline-v2/orchestrator.ts`, line 582):

```

: await runWithTimeout("1_ingestion", () => runIngestionStage(ctx)).catch((err) => {
  const isTimeout = err instanceof StageTimeoutError;
  const reason = isTimeout
    ? `stage_timeout: exceeded ${err.budgetMs}ms budget`
    : `engine_failure: ${String(err)}`;
  ctx.log("Stage 1", `${isTimeout ? "TIMEOUT" : "ERROR"}: ${err.message} - producing empty doc`);
  return {
    status: "degraded" as const,
    data: { documents: [], primaryDocumentIndex: -1, totalDocuments: 0 },
    error: err.message,
    // ...
  };
});

```

The key elements of a compliant block are:

1. The error is classified (timeout vs engine failure).
2. The error is logged with context (stage name, error message, reason).
3. A degraded result is returned rather than null or undefined.
4. The `status: "degraded"` field signals to downstream stages that this stage's output is incomplete.

In tRPC procedures, errors are surfaced as `TRPCError` with an appropriate code (`UNAUTHORIZED` , `FORBIDDEN` , `NOT_FOUND` , `BAD_REQUEST` , `INTERNAL_SERVER_ERROR`). The code determines the HTTP status code returned to the client. Never throw a plain `Error` from a tRPC procedure — always throw a `TRPCError` .

The one legitimate exception to the no-silent-catch rule is fire-and-forget side effects (e.g., the tenant isolation violation logger, the benchmark learning write). These catch errors and write them to `console.error` only, because a logging failure must never block the primary operation. This exception is documented at the call site.

Extraction Robustness

When OCR or document extraction partially fails, the pipeline behaves as follows:

1. The stage input guard (`server/pipeline-v2/stageInputGuards.ts`) detects missing or low-confidence fields and produces a `DATA_GAPS_WARNING` preamble.
2. The preamble is prepended to the LLM system prompt for the affected stage.
3. The LLM is instructed to mark missing fields as “insufficient data” rather than inferring values.
4. The stage output includes an `assumptions` array listing every field that was defaulted or inferred, with the reason.
5. The orchestrator stores the `StageInputReport` on `ctx.stageInputReports[stageKey]` for inclusion in the final report.

A claim with critical extraction gaps will produce a degraded assessment, not a failed pipeline. The report will explicitly list the data gaps and reduce the confidence score accordingly.

Scoring Consistency

“Consistent” in the KINGA context means:

- **Idempotent on re-run:** Running the pipeline twice on the same claim with the same documents produces the same scores. The orchestrator's partial-resume mechanism (`loadCompletedStages` , `server/pipeline-`

v2/orchestrator.ts , line 492) restores completed stage outputs from the database on re-run, ensuring that completed stages are not re-executed with potentially different LLM outputs.

- **Deterministic given identical inputs:** The physics engine (M1, M4, M5, M6) is deterministic. The LLM-based stages (M7 excluded) are non-deterministic by nature, but the partial-resume mechanism ensures that a re-run does not re-execute LLM stages that have already completed successfully.
- **Verifiable:** Every pipeline run produces a pipeline_runs row and per-stage pipeline_stage_results rows. These can be queried to verify what inputs were used and what outputs were produced for any given run.

To verify scoring consistency for a specific claim, query pipeline_stage_results WHERE run_uuid = ? and compare the output_json for each stage across two run UUIDs for the same claim.

12. Testing and QA Requirements

What Must Be Tested Before Any Change

The following five areas require test coverage before any change touching them is merged. The current coverage status is noted for each.

Area	Required Coverage	Current Status
Physics pipeline	Speed inference ensemble (M1–M7 methods, consensus algorithm, anti-circularity), integrity checks (INT-01 to INT-10), VGE scale calibration	Gap: No dedicated test for <code>speedInferenceEnsemble.ts</code> . <code>server/physics-types.test.ts</code> and <code>server/physicsAndStage26Fixes.test.ts</code> cover type contracts and specific fixes but not the full ensemble logic.
Currency path	All currency-bearing values carry explicit currency codes; NCI normalisation; USD conversion; multi-currency claim processing	Covered: <code>server/currency.test.ts</code> , <code>server/claim-currency.test.ts</code> , <code>server/cost-extraction-currency.test.ts</code>
Tenant isolation	<code>requireInsurerDomain</code> middleware rejects cross-tenant requests; violation logging writes to <code>tenant_isolation_violations</code> ; admin bypass is logged but not recorded as violation	Covered: <code>server/tenant-isolation.test.ts</code> , <code>server/batch2-tenant-isolation.test.ts</code> , <code>server/rgh16-tenant-isolation.test.ts</code> , <code>server/tenant-isolation-violation-logging.test.ts</code> , <code>server/marketplace-tenant-isolation.test.ts</code>
Report generation	Report HTML does not include application chrome; PDF renderer produces output for each report tier; <code>REPORT_ACCESS</code> map correctly gates access by role	Partially covered: <code>server/reporting.test.ts</code> , <code>server/kinga-reports.test.ts</code> , <code>server/reports.test.ts</code> , <code>server/report-signals.test.ts</code> . Gap: No test verifying PDF output is free of application chrome.
Benchmark write path	G-1 fraud guard prevents writes for <code>fraud_score ≥ 50</code> ; automatic finalization write triggers at <code>ANALYSIS_COMPLETE</code> ; adjuster correction UPSERT overwrites finalization record	Gap: No test covers <code>component_repair_outcomes</code> writes. <code>server/pipeline-v2/validatedOutcomeRecorder.test.ts</code> and <code>server/pipeline-v2/costLearningRecorder.test.ts</code> exist but do not test <code>recordAdjusterOutcome()</code> or the automatic finalization write path.

Cross-Validation as a QA Layer

The cross-validation engine (`server/cross-validation.ts`) is a QA layer that compares quoted parts against damage visible in photographs. To run it manually against a specific claim:

1. Retrieve the claim's `quotedParts` array (from the panel beater quotation in `ai_assessments.costAnalysisJson`).
2. Retrieve the claim's `damagePhotoUrls` array (from `claims.damagePhotoUrls` or `ai_assessments.physicsAnalysisJson`).
3. Call `crossValidateQuotesVsPhotos(quotedParts, damagePhotoUrls)` directly in a Node.js script.
4. Inspect the `CrossValidationReport` for `confirmedCount`, `quotedNotVisibleCount`, `visibleNotQuotedCount`, and `overallRiskScore` .

A **false pass** occurs when photo analysis returns low confidence (blurry, dark, or low-resolution photos) and the engine defaults to “confirmed” for all quoted parts. The `photoMetadataScore` signal in the fraud input captures this scenario. If `photoMetadataScore` is below 0.5 and `confirmedCount` equals the total quoted parts count, the cross-validation result should be treated as inconclusive rather than clean.

13. Deployment and Release Process

Current Process

The deployment process is managed through GitHub Actions (`.github/workflows/cicd-pipeline.yml`) and the Manus Management UI. The pipeline has five stages:

Stage	Trigger	Gate Type
Code Complete	Push to <code>main</code> or <code>develop</code> , or PR	Soft gate — TypeScript check, ESLint
Peer Review	PR only	Hard gate — minimum 2 approvals required
Automated Tests	After Code Complete	Hard gate — <code>pnpm test</code> must pass (soft-fail removed in Batch 1 remediation)
Stability Gates (G1–G7)	After Automated Tests	G1 (regression) is hard; G4 (DB integrity) is currently soft
Release Candidate Build	After Stability Gates, <code>main</code> branch only	Produces build artifacts; creates timestamped RC tag

Production deployment is **manual**. The CI pipeline notifies that a release candidate is ready, and the engineer must click the Publish button in the Manus Management UI. There is no automated production push.

Database Migrations

Schema changes are managed with Drizzle Kit. The workflow is:

1. Edit `drizzle/schema.ts` to add or modify tables and columns.
2. Run `pnpm db:push` (which runs `drizzle-kit generate && drizzle-kit migrate`).
3. Verify the migration applied correctly by querying the database.

`pnpm db:push` applies migrations to the **live database** (the `DATABASE_URL` environment variable). There is no separate staging database in the current setup. This means schema changes are applied directly to production. Engineers must exercise extreme caution with destructive migrations (dropping columns, changing column types). Database data is not recoverable from the platform tooling — always take a backup before a destructive migration.

Environment Variables

All secrets and environment variables are managed through the Manus Management UI (Settings → Secrets). They are injected into the runtime at startup. The following variables are required for the platform to function:

Variable	Purpose
<code>DATABASE_URL</code>	MySQL/TiDB connection string
<code>JWT_SECRET</code>	Session cookie signing
<code>VITE_APP_ID</code>	Manus OAuth application ID
<code>OAUTH_SERVER_URL</code>	Manus OAuth backend base URL
<code>BUILT_IN_FORGE_API_URL</code>	Manus built-in APIs (LLM, storage, etc.)
<code>BUILT_IN_FORGE_API_KEY</code>	Bearer token for server-side API calls

What Should Eventually Be Automated

The following steps are currently manual and should be automated in a future release:

- Production deployment (currently requires a manual Publish button click).
- Database backup before destructive migrations.
- G4 (DB integrity gate) — currently soft-fail; should be a hard gate once the foreign key check script is complete.
- Post-deployment smoke test — a synthetic claim submission that verifies the pipeline runs end-to-end in production.

14. Onboarding Path for a New Engineer

First-Week Reading Sequence

A new engineer should read the following files in this order, spending approximately one hour on each group before moving to the next.

Day 1 — System orientation

1. This manual (all sections).
2. `docs/KINGA_END_TO_END_WALKTHROUGH_v1.0.md` — traces a single vehicle through eight complete platform scenarios. Read Scenarios 1, 4, and 5 in detail; skim the rest.
3. `drizzle/schema.ts` — the canonical data model. Focus on `claims`, `ai_assessments`, `component_repair_outcomes`, and `workflow_audit_trail`.
4. `server/rbac.ts` — roles, workflow states, and the legal transition graph.

Day 2 — Pipeline internals

1. `server/pipeline-v2/orchestrator.ts` — read the file header (lines 1–430) which documents the full stage map, parallel execution points, and design contracts. Do not read the full implementation yet.
2. `server/pipeline-v2/types.ts` — the `Stage10output` through `Stage90output` types. These are the contracts between stages.
3. `server/pipeline-v2/speedInferenceEnsemble.ts` — the M1–M7 speed inference methods. Read the file header comments carefully (lines 1–100).
4. `server/pipeline-v2/stage-integrity.ts` — the 10 physics integrity checks.

Day 3 — Governance and safety

1. `server/_core/trpc.ts` — the `requireInsurerDomain` middleware and tenant isolation enforcement.
2. `server/workflow-engine.ts` — the single gateway for all claim state transitions.
3. `shared/fraudScoring.ts` — the canonical fraud score bands.
4. `docs/KINGA-FRAUD-SCORING-STANDARD.md` — the governance standard for fraud score bands.

Day 4 — Safe end-to-end walkthrough

Use the following test claim for a safe end-to-end walkthrough. This claim was used in the KINGA-INV-001 investigation and is confirmed to have a low fraud score ($10/100$), making it safe to use for learning without triggering fraud escalation workflows.

Test claim reference: CLM-TEST-1785489890585 (claim ID 11590068, assessment ID 16410008)

Vehicle: Toyota Fortuner 2.8 GD-6, 2021, White

Impact type: Frontal collision (moderate severity)

Fraud score: $10/100$ (minimal risk)

Do not use claims with fraud scores above 40 for learning walkthroughs, as they may trigger escalation notifications to the fraud investigation unit.

Day 5 — Bug history review

Before touching the pipeline unsupervised, a new engineer must be able to explain the following three incidents from memory:

1. **The unactivated benchmark learning loop** (KINGA-INV-001, 2026-07-31): The `component_repair_outcomes` table had 0 rows in production because the write path was gated behind a UI action with no caller. The lesson: any pipeline side-effect that depends on a UI trigger must have an automated fallback trigger, or a test that fails if the UI path is never exercised.
2. **The M1/M5 crush-depth correlation bug** (KINGA-AUDIT-2026-07): M1 (Campbell) and the original M5 Path A both used the same `crushDepthM` input. The prior weights (0.45 for M5) overstated independence, effectively double-counting the crush depth measurement. The fix was to reduce M5's weight to 0.10 and remove M5 Path A. The lesson: when two methods share a common input, their weights in a consensus must reflect their actual independence, not their theoretical independence.
3. **The circular crush-depth validation bug class** (documented in `server/pipeline-v2/damageClassificationEngine.ts`, lines 308–338): Comparing crush depth against a range derived from the consensus speed is circular — the check always passes because the crush depth was used to compute the speed. The fix was to anchor the check on the Stage 6 severity/structural rating, which is independent of the crush depth measurement. The lesson: any validation logic must not depend on a value derived from the thing it is validating.

15. Glossary

Every acronym and domain term used in this manual is defined here.

Term	Definition
ADR	Architecture Decision Record — a document recording a significant architectural decision, its context, and its rationale. KINGA's ADR library is at <code>docs/KINGA_ADR_LIBRARY_v1.0.md</code> .
ANALYSIS_COMPLETE	The pipeline status value written to <code>claims.status</code> when the full pipeline has completed successfully. Triggers the automatic benchmark learning write.
Campbell's Formula	A biomechanics formula relating vehicle crush depth to impact speed via vehicle stiffness coefficients. The basis for M1 in the speed inference ensemble. Source: Campbell (1974), SAE 740565.
Claimant-Stated Speed	The impact speed stated by the driver in the claim documents. Stored in <code>claims.claimantStatedSpeedKmh</code> . Immutable after first write. Never overwritten by pipeline-derived values.
Consensus Speed	The weighted-average impact speed produced by the speed inference ensemble (M1–M7). Stored in <code>claims.estimatedSpeedKmh</code> . Distinct from the claimant-stated speed.
Cross-Validation	The QA process of comparing quoted repair parts against damage visible in photographs to detect scope creep or phantom parts. Implemented in <code>server/cross-validation.ts</code> .
DATA_GAPS_WARNING	A plain-English preamble prepended to an LLM system prompt when stage input fields are missing or low-confidence. Produced by <code>server/pipeline-v2/stageInputGuards.ts</code> .
FEL	Forensic Execution Ledger — an append-only log of every significant decision and evidence point in a pipeline run, used for audit and dispute resolution.
FMVSS 208	Federal Motor Vehicle Safety Standard 208 — the US standard governing occupant crash protection, including airbag deployment thresholds. Referenced in M4 of the speed inference ensemble.
G-1 Guard	The fraud-risk exclusion guard that prevents claims with <code>fraud_risk_score ≥ 50</code> from writing to <code>component_repair_outcomes</code> . Implemented in <code>server/pipeline-v2/repairReplaceEngine.ts</code> .
INT-01 to INT-10	The ten physics integrity checks performed by the Physics Integrity Engine (<code>server/pipeline-v2/stage-integrity.ts</code>).
M1–M7	The seven speed inference methods in the speed inference ensemble. M2 and M3 are disabled. See Section 5 for the full table.
NCI	National Cost Index — a regional cost adjustment factor applied to repair cost benchmarks to account for purchasing power differences between countries. Implemented in the Economic Context Engine in <code>server/db.ts</code> .
OCR	Optical Character Recognition — the process of extracting text from scanned documents or images. Used in Stage 2 of the pipeline.
PhysicsTruth	The canonical physics output object produced by Stage 7. Contains the consensus speed, all method estimates, the integrity engine result, and the claimant-speed

Term	Definition
	deviation flag. Stored in <code>ai_assessments.physicsTruthJson</code> .
Pipeline Run UUID	A unique identifier for a single pipeline execution. Links the <code>claims</code> row to its <code>pipeline_runs</code> row and all <code>pipeline_stage_results</code> rows.
PRE_ASSESSMENT / POST_ASSESSMENT	The two operating modes of the cost optimisation engine. PRE_ASSESSMENT uses the AI estimate as a benchmark; POST_ASSESSMENT uses the assessor's validated cost as the primary basis.
RBAC	Role-Based Access Control — the system by which access to procedures, data, and reports is controlled by the user's assigned role. Implemented in <code>server/rbac.ts</code> and <code>server/_core/trpc.ts</code> .
SAE 2002-01-0547	Varat & Husher (2002) — the academic paper providing the crash energy signature data used in M6 (Severity-Anchored Inference).
SLA	Service Level Agreement — the time limit within which a claim must be processed. Default is 72 hours for intake queue claims. Displayed via <code>SLADeadlineChip</code> .
SLPE	Structural Load Path Estimator — the sub-engine (<code>server/pipeline-v2/stage-6-5c-slpe.ts</code>) that estimates component-level deformation energy from the structural load path.
SOD	Segregation of Duties — the governance requirement that no single user can complete the full claim lifecycle. Enforced by the workflow engine via <code>claim_involvement_tracking</code> .
Stage N	A numbered processing stage in the pipeline orchestrator. Stages 1–5 handle document ingestion and extraction; Stage 6 handles damage analysis; Stage 7 handles physics; Stages 8 and 9 handle fraud and cost; Stage 10 handles report generation.
trRPC	TypeScript Remote Procedure Call — the type-safe API layer used throughout KINGA. Procedures are defined in <code>server/routers.ts</code> and consumed via <code>trpc.*</code> hooks on the frontend.
Tenant	An insurer organisation using the KINGA platform. All claim data is scoped to a tenant via <code>tenantId</code> . Tenant isolation is enforced at the API layer by <code>requireInsurerDomain</code> .
VGE	Vision Geometry Engine — the sub-engine (<code>server/pipeline-v2/stage-6-5a-vge.ts</code>) that converts pixel measurements in damage photographs to physical dimensions using reference objects.
VGR	Vision Geometry Reconciler — the sub-engine (<code>server/pipeline-v2/stage-6-5b-vgr.ts</code>) that produces a cross-image crush depth consensus from multiple photographs.
VIN	Vehicle Identification Number — the unique 17-character identifier for a vehicle. A VIN mismatch between the claim documents and the vehicle registry is a fraud signal.

Term	Definition
VVCS	Vehicle-centric Coordinate System — the coordinate convention used by the VGE: origin at front axle centreline at ground level, X positive forward, Y positive left, Z positive up, all in millimetres.
Workflow Engine	The single gateway for all claim state transitions (<code>server/workflow-engine.ts</code>). All transitions must go through <code>transitionClaim()</code> .
WorkflowState	The governance-layer claim state enum defined in <code>server/rbac.ts</code> . Distinct from the legacy <code>status</code> enum. Both are written on every transition for backward compatibility.

16. Gaps and Open Questions

This section documents everything that could not be confirmed from the code with confidence, areas where the code and existing documentation disagree, and areas with no test coverage that this manual implies should have some.

Confirmed Test Coverage Gaps

Gap	Area	Risk
No test for <code>speedInferenceEnsemble.ts</code>	Physics pipeline	High — the speed inference ensemble is the most complex deterministic engine in the platform. The M1/M5 weight correction (KINGA-AUDIT-2026-07) has no regression test. A future weight change could reintroduce the double-counting error without detection.
No test for <code>component_repair_outcomes</code> write path	Benchmark learning loop	High — the G-1 fraud guard and the automatic finalization write path have no test coverage. The historical failure (0 rows in production) was caused by an unactivated write path; without a test, a future refactor could silently deactivate it again.
No test for PDF chrome isolation	Report generation	Medium — there is no test verifying that report HTML output does not include application chrome. This is an invariant (INV-08) with no automated enforcement.
No test for <code>cross-validation.ts</code> false-pass scenario	QA layer	Medium — the false-pass scenario (low-confidence photos causing all parts to be “confirmed”) is documented but not tested.

Code and Documentation Disagreements

Disagreement	Location	Status
<code>fraud-scoring.ts</code> uses different thresholds than <code>shared/fraudScoring.ts</code>	<code>server/fraud-scoring.ts</code> <code>rawRiskLevel()</code> (76/56/36/16) vs <code>shared/fraudScoring.ts</code> (81/61/40/20)	Known inconsistency. The canonical thresholds are in <code>shared/fraudScoring.ts</code> . The <code>rawRiskLevel()</code> function in <code>fraud-scoring.ts</code> is a legacy implementation that pre-dates the governance standard. Any new code must use <code>shared/fraudScoring.ts</code> .
<code>risk_manager_portfolio</code> key does not match dot-notation convention	<code>server/reporting/reportDefinitions.ts</code>	TD-004 in the v1.0 baseline certification report. Scheduled for correction in v1.1.
Deployment guide describes Kafka/Kubernetes architecture	<code>docs/deployment-guide.md</code>	The deployment guide (authored February 2026) describes a Kafka-based microservices architecture with Kubernetes. The actual production deployment is a monolithic Node.js application on Manus Autoscale (Cloud Run). The deployment guide is outdated and should not be used as a reference for the current deployment process.
<code>FraudRiskLevel</code> type alias	<code>server/pipeline-v2/types.ts</code>	TD-001 in the v1.0 baseline certification report. The type alias exists but has no runtime impact.

Unconfirmed Behaviours

Item	What Could Not Be Confirmed
Pipeline resume on crash	The partial-resume mechanism (<code>LoadCompletedStages</code>) is implemented in the orchestrator, but it was not possible to confirm from the code alone that it correctly handles all stage failure modes (e.g., a stage that wrote partial output before crashing).
NCI calibration data source	The NCI model in <code>server/db.ts</code> applies regional cost adjustments, but the source data for the NCI values (the actual index values per country) was not located in the codebase. It may be hardcoded in the Economic Context Engine or loaded from a configuration table.
WhatsApp API integration	The high-level data flow diagram (Section 1) lists WhatsApp as a claim source, but no WhatsApp integration code was located in the codebase during this audit. This may be a planned integration not yet implemented.
Fleet portal routing	The fleet portal is listed in the portal inventory (Section 3), but no dedicated <code>FleetPortalLayout.tsx</code> component was located. Fleet claims may be handled through the standard insurer portal with fleet-specific filtering.